

EMULATED_DEVICE_TESTING

Helix OTA — Emulated / Virtual Device Testing Strategy

Field	Value
Revision	2
Last modified	2026-06-11T10:30:00Z
Status	active — Tier-1 shipped; Tier-1.5 dev-host A/B-virt foundation built, slot mechanism in progress
Status summary	The tiered plan for exercising the OTA stack against device-shaped targets without (Tier-1) and with (Tier-1.5 / Tier-2 / Tier-3) real A/B slot-switch + dm-verity + hardware. Tier boundaries are FACT, established from the host's available runtimes and the containers submodule's capabilities — not guesses. Now built: Tier-1 (T0 protocol emulator, shipped) plus a new Tier-1.5 dev-host A/B-virt tier (QEMU virt+HVF + U-Boot bootcount/altbootcmd + RAUC dm-verity) whose FOUNDATION boots to live userspace on this Apple-Silicon host (PROVEN); its real A/B slot switch / dm-verity / auto-rollback is in progress , gated on the in-flight u-boot.bin build — NOT proven (§11.4.6). Tier-2 Cuttlefish stays host-gated (no KVM on macOS).
Authority	Helix OTA control-plane / device-integration team
Related	docs/research/main_specs/CONTINUATION.md; docs/RESUMPTION.md; containers/ submodule (vasic-digital/containers, §11.4.76); submodules/ota-protocol, submodules/ota-android-agent, submodules/ota-update-engine-bridge

1. Purpose

The control plane (server/), the ota-protocol wire contract, and the device-side decision logic (ota-android-agent DeltaApplyDecision, ota-update-engine-bridge) must be validated against device-shaped targets. The realistic targets — RK3588 / Orange Pi 5 Max — are not always available, and the real Android A/B update path (update_engine + AVB/dm-verity + auto-rollback) cannot run on the current Apple-Silicon development host. This doc records the

tiered plan that lets each layer be exercised at the highest fidelity the available environment allows, with an honest boundary on what each environment can and cannot prove.

2. Tiers (FACT)

Tier-1 — protocol-level device emulator (runnable on THIS macOS host NOW)

- **What:** a podman container running a Go `ota-device-emulator` that speaks the **real ota-protocol** to a live control plane and walks the full OTA lifecycle: **register** → **update-check** → **telemetry** → **delta** → **rollout** → **recall**.
- **Why it is real (not a mock):** the emulator uses the actual `ota-protocol` types and the actual `HTTP /api/v1` surface — it is a real client of the real server, not an in-process fake. Per §11.4.27, integration/e2e tiers exercise the real, fully implemented system; the emulator stands in only for the device's *physical* layer (no real `update_engine`, no real partitions), which Tier-1 explicitly does not claim to cover.
- **What it proves:** wire-contract conformance, server-side flow correctness (anti-downgrade invariant, delta selection + full fallback, rollout cohort advancement, recall = forward-fix), telemetry ingest/read round-trips, multi-device fleet behaviour under concurrent emulated devices.
- **What it does NOT prove (honest boundary):** real A/B slot switch, dm-verity verification on real partitions, auto-rollback on a corrupt slot, vendor HAL, U-Boot bootloader slot selection. The U-Boot slot-switch + dm-verity portion is now addressed on the dev host by **Tier-1.5** below; the Android `update_engine` + AVB portion remains Tier-2/3.
- **Runtime:** podman on this host (the same runtime the `pgx` integration + dev stack already use via the `containers` submodule). No KVM, no QEMU, no AVD required.
- **Status: shipped** — full-lifecycle + multi-device fleet + recall→recovery e2e proven, captured transcripts under `docs/qa/`.

Tier-1.5 — dev-host A/B-virt (U-Boot + RAUC on QEMU virt + HVF) — built; slot mechanism in progress

- **What:** a generic aarch64 Linux guest (Buildroot base + kernel) booted under **QEMU -machine virt + HVF** on this Apple-Silicon host, carrying a real **U-Boot bootcount/altbootcmd A/B slot-switch + RAUC dm-verity A/B update client** over a 2-slot GPT disk — a real slot switch + auto-rollback WITHOUT a KVM gate. This sits between Tier-1 (protocol only) and Tier-2 (Android `update_engine`): it is NOT the RK3588 SoC and NOT Android's `update_engine`, but it IS a genuine bootloader-driven A/B/dm-verity/rollback exercise runnable locally. See [../research/rk3588_emulator/REPORT.md](#) §3 and `tests/emulator/ab_virt/`.
- **What it proves (target):** real U-Boot slot selection, bootcount-driven auto-rollback, RAUC dm-verity verification of the inactive slot — the bootloader/verity half of the apply path that Tier-1 stubs.

- **What is DONE (PROVEN, captured evidence):**
 - **PWU-AB-0** — base aarch64 Buildroot image built (tests/emulator/ab_virt/build_image.sh → out/images/{Image, rootfs.ext2}), committed.
 - **PWU-AB-1 FOUNDATION** — the base image **boots to a live interactive userspace on QEMU + HVF**, PROVEN by the captured boot console docs/qa/20260611T061626Z-ab-virt-boot/console.log (HELIX_USERSPACE_LIVE_OK).
 - HelixQA bank challenge H0TA-AB-VIRT-B00T wired to the boot test.
- **What is IN PROGRESS / PENDING (NOT proven, §11.4.6):** the real A/B slot switch / dm-verity / auto-rollback itself. The 2-slot GPT disk assembler (assemble_ab_disk.sh) + U-Boot boot.cmd/env (uboot_ab/) are **authored** (parse-clean + coherent) but **NOT yet run** — they are gated on the in-flight U-Boot + RAUC build producing u-boot.bin (the build is running; out/images/ currently holds the kernel + rootfs but no u-boot.bin yet). Until that disk boots and a switch is observed (findmnt / + cat /etc/slot_id across a switch + rollback trace), the slot mechanism is recorded as **NOT proven**, never as a green.
- **Runtime:** QEMU aarch64 virt + HVF on this macOS host; guest image + GPT disk assembled inside a named podman aarch64 Linux container (this host is macOS).
- **Status: foundation shipped + proven; A/B slot mechanism in progress** (gated on u-boot.bin).

Tier-2 — Cuttlefish virtual Android device (host/hardware-gated)

- **What:** a Cuttlefish (cvd) virtual Android device running a real Android system image, exercising the **real update_engine A/B flow + AVB/dm-verity + auto-rollback** end-to-end, driven by the ota-android-agent + ota-update-engine-bridge.
- **What it proves:** the device-side apply path the Tier-1 emulator stubs — real payload application to the inactive slot, post-reboot slot promotion, and corrupt-slot → auto-rollback.
- **Gate (FACT):** Cuttlefish requires **Linux with nested KVM**. The current development host is Apple-Silicon using the applehv hypervisor, which **cannot** run Cuttlefish. Tier-2 therefore requires a Linux CI runner / Linux box with nested KVM.
- **Honest §11.4.112 boundary:** Tier-2 is **host/hardware-gated, NOT structurally impossible**. It runs the moment a Linux + nested-KVM environment is available; the blocker is environment provisioning, not a platform/protocol impossibility.
- **Harness state:** the Cuttlefish A/B harness is **authored** (tests/emulator/tier2_cuttlefish_ab.sh), including the **corrupt-slot → reboot → auto-rollback** section (PWU-CF-2). On this macOS dev host the whole script **SKIPS-with-reason at the topology gate** (no /dev/kvm) — it is NOT run and NOT proven here.
- **Status:** harness authored; **host-gated SKIP on macOS** — unproven until run on a Linux + nested-KVM host.

Tier-3 — real RK3588 / Orange Pi 5 Max hardware

- **What:** the real target board(s) — full vendor HAL, real **U-Boot slot-switch**, **dm-verity on real partitions**, real update_engine, real auto-rollback on a genuinely corrupt slot.
- **What it proves:** everything Tier-2 proves plus the vendor-specific bootloader and hardware-backed verification that only the physical SoC exercises.
- **Gate (FACT):** requires the physical RK3588 / Orange Pi 5 Max board (operator hardware), per §11.4.133 target-hardware-safety discipline for any flash.
- **Honest §11.4.112 boundary:** hardware-gated, NOT structurally impossible.
- **Status:** pending hardware availability (the CONTINUATION NEXT-wave items 1–2 land here).

3. Tier coverage matrix

Legend: **YES** = proven with captured evidence · *target* = the tier's intended scope, NOT yet proven · no = out of scope for that tier.

Capability	Tier-1 (podman emulator)	Tier-1.5 (A/B- virt, U- Boot+RAUC)	Tier-2 (Cuttlefish)	Tier-3 (RK3588)
ota-protocol wire conformance	YES	(host plumbing)	YES	YES
Server flow: register/update- check/telemetry/ delta/rollout/ recall	YES	(host plumbing)	YES	YES
Anti-downgrade invariant	YES	(host plumbing)	YES	YES
Boots to live userspace on QEMU+HVF	n/a	YES (PWU- AB-1 foundation)	n/a	n/a
Real U-Boot bootcount A/B slot-switch	no	<i>target — in progress (authored, gated on u-boot.bin)</i>	no	YES
RAUC dm- verity verification	no	<i>target — in progress</i>	no (AVB instead)	YES
Auto-rollback on corrupt slot	no	<i>target — in progress</i>	<i>target (host- gated SKIP on macOS)</i>	YES
Real Android update_engine A/B apply	no	no (U-Boot/ RAUC, not update_engine)	<i>target (host- gated)</i>	YES
AVB verification	no	no (RAUC dm- verity instead)	<i>target (virtual, host-gated)</i>	YES (real partitions)

Capability	Tier-1 (podman emulator)	Tier-1.5 (A/B- virt, U- Boot+RAUC)	Tier-2 (Cuttlefish)	Tier-3 (RK3588)
Vendor HAL / RK3588 SoC bootloader	no	no	no	YES
Runnable on this macOS host now	YES	YES (foundation booted; slot disk pending u- boot.bin)	no (Linux+KVM)	no (hardware)

4. Reuse of the containers submodule (§11.4.76, extend-don't-reimplement §11.4.74)

The vasic-digital/containers submodule (digital.vasic.containers) is the canonical containerization machinery and already provides the primitives each tier needs:

- **pkg/boot + pkg/compose + pkg/health** — on-demand boot, compose orchestration, and readiness gating. Tier-1's podman emulator + control plane stack boot via these (the same path the pgx integration test already uses), satisfying the §11.4.76 on-demand-infra invariant (the test entry point boots the infra; operators never start podman by hand).
- **pkg/emulator** — multi-target Android emulator orchestration (AVD, x86_64, with KVM acceleration gating, AVD-lock clearing, orphan qemu-system-* reaping). Relevant to an x86_64-AVD variant of Tier-2 on a Linux/KVM host.
- **pkg/vm** — QEMU VM orchestration (**aarch64** via `-machine virt + AAVMF` `UEFI`), the seam Tier-1.5 builds on for its U-Boot + RAUC A/B-virt guest (QEMU virt + HVF on this macOS host).

Tier-1.5 reuse (§11.4.74): the dev-host A/B-virt tier reuses **U-Boot** (bootcount/altbootcmd slot-select) and **RAUC** (dm-verity A/B update client) upstream — neither reimplemented in-project — on top of pkg/vm's QEMU aarch64 virt + HVF boot. The guest image + 2-slot GPT disk are assembled inside a named podman aarch64 Linux container (tests/emulator/ab_virt/).

Extension policy: where a tier needs a primitive the submodule lacks (e.g. a Cuttlefish cvd lifecycle wrapper, or an OTA-specific device-emulator harness), we **extend the submodule upstream via PR** per §11.4.74 — never duplicate the machinery in-project. The Tier-1 ota-device-emulator itself is OTA-domain logic (it speaks ota-protocol), so it lives in the project; the boot/health/compose plumbing around it is the submodule's job.

5. Anti-bluff posture (§11.4 / §11.4.27 / §11.4.69)

Each tier produces captured evidence under docs/qa/<run-id>/: Tier-1 captures the real request/response transcripts for the full lifecycle against a live server; Tier-2/3 add the on-device apply/rollback evidence. A tier claiming a PASS without

exercising the real system at that tier's fidelity is a §11.4 PASS-bluff. The tier boundaries above are stated as FACT precisely so no tier silently claims coverage it does not have.